A CODE-GROUNDED EXPLANATION / SPEAKING SCRIPTS + DEMO Q&A;

Medix: presentation prep guide

What the project does, why each component exists, and how to explain the architecture

This guide explains the implemented Medix system in plain language, then connects each term to a concrete request, code module and demonstration. Use the boxed scripts to rehearse; use the question bank to prepare for discussion.

> **Suggested speaking script**
> Medix is a medical-assistance research prototype that connects a user's question to relevant instructional video evidence and a small set of checked medical references. It combines retrieval, video localization and language-model generation, then checks whether the answer is supported. The goal is a useful conversational answer with inspectable support, rather than simply returning a transcript or an unrelated video.

Part	Pages	What to study
A. The project and graph	2-3	Problem, solution and the overall architecture diagram
B. Explain every component	4-15	Next.js, FastAPI, HTTP endpoints, evidence, generation, references, Voyager, retrieval and three input modes
C. Rehearse the meeting	16-20	Three demo cases, five-minute run of show and likely professor questions
D. Quick reference	21-24	Terms, source map, editable Mermaid diagrams and exact launch commands

How to use this guide

- For a quick rehearsal, read pages 2-3, the scripts on pages 4-10, and the timed outline on page 17.
- For the demo, use page 16 and keep pages 18-20 ready for questions. Page 24 contains the launch commands.

Scope: based on the active code at /Users/sairithwikk/Documents/Medix, inspected on 15 October 2026. The supplied presentation guide influenced organization and formatting only; its paper, methods and results are not part of Medix. No application code was changed.

MEDIX / PRESENTATION PREPARATION

A1. The problem and the proposed solution

What problem is Medix trying to address?

A medical-procedure video can be useful, but a user may need one answer from a long recording: how to check a bandage, what a particular step means, or what to do next. Manually finding the relevant moment is slow. A plain chatbot answer is easier to read but may leave the user unable to inspect where the information came from.

A simple retrieval system also has limitations: a video title may match the general topic while its selected transcript passage does not answer the actual question. Copying that passage produces fragmented or incomplete instructions. Follow-ups add another difficulty because words such as "their", "it" or "next" depend on the earlier conversation.

Observed problem	How the implementation addresses it
Long videos and broad title matches	Search video metadata and transcript passages, then localize an interval using transcript and sampled-frame relevance.
Robotic transcript-style answers	A conversational response layer generates a direct answer from selected evidence and applicable checked references.
Follow-ups lose the situation	Keep a recent conversation summary and the active evidence bundle; reuse it for same-topic follow-ups.
Irrelevant moments or confusing citations	Select directly relevant cues and separate video timestamps from medical-reference links.
Unsupported medical detail	Validate citations and claim support; check relevance, scope and completeness; allow a bounded repair or partial answer.

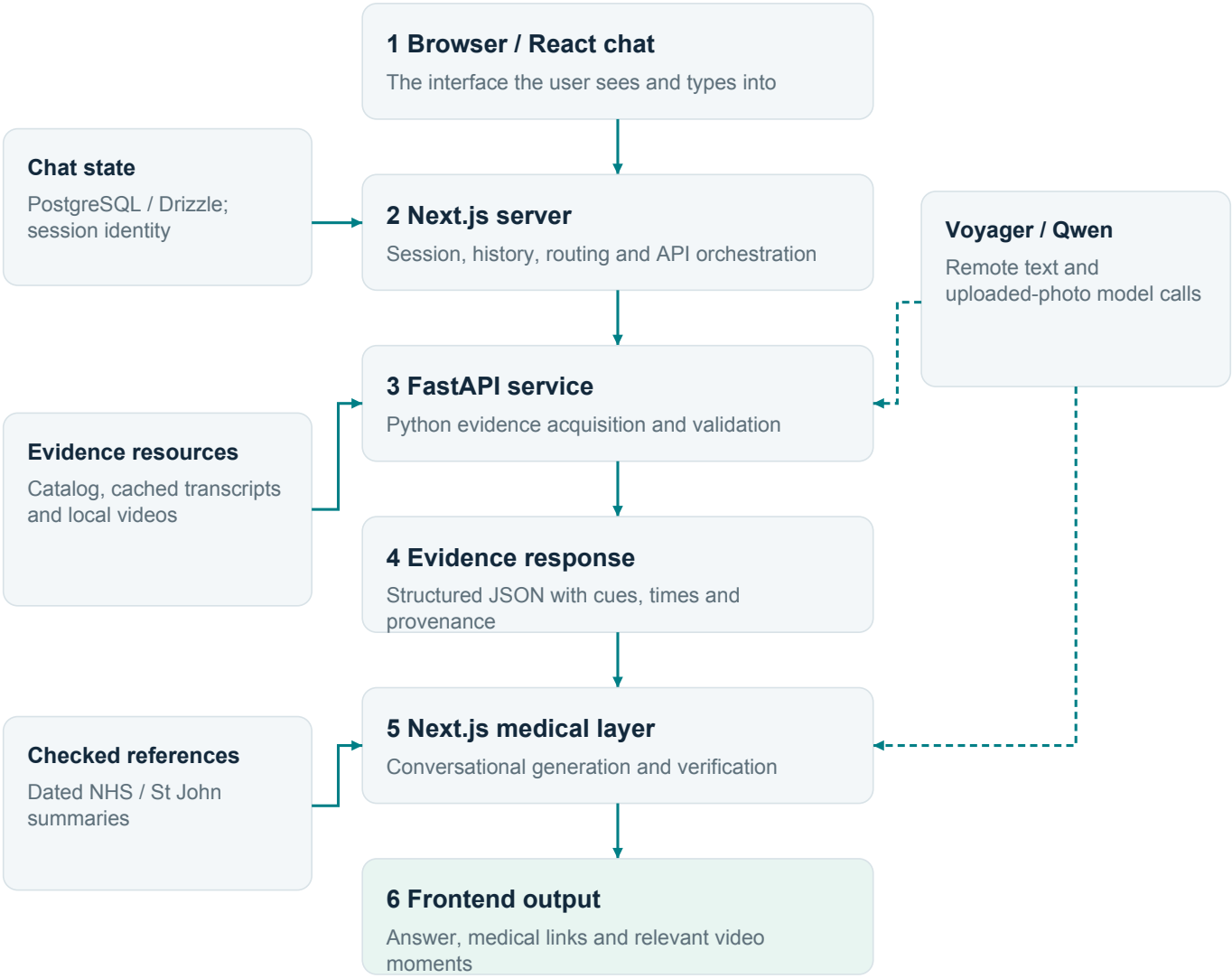
Suggested speaking script
The problem is not simply answering a medical question. It is answering it clearly while showing useful evidence. Medix searches for the relevant part of a medical video, constructs an evidence package, and then turns that material into a conversational explanation. The recent work targets transcript copying, unrelated evidence and weak follow-ups. These changes address specific failure modes; they do not establish that the system is clinically reliable in every situation.

One sentence to remember
Medix connects a medical question to a readable answer and inspectable supporting evidence.

MEDIX / PRESENTATION PREPARATION

A2. Read the architecture graph aloud

Read the center from top to bottom. Side boxes supply stored information or model inference. "Response layer" means application code, not a separate physical server. The arrows are data handoffs, not model training.



Suggested speaking script
The browser sends the message to the Next.js server. Next.js decides whether to clarify, reuse existing evidence, or ask the Python backend for new evidence. FastAPI returns a structured evidence response. The medical response code inside Next.js uses that evidence, checked references and conversation history to generate and verify the user-facing answer. Voyager supplies remote model inference for both services.

Edit this diagram in Mermaid Live | 01-overall.mmd

Same-topic follow-ups can skip a new FastAPI request. Photo-only input exits earlier with a clarification. New images require fresh retrieval.

MEDIX / PRESENTATION PREPARATION

B1. What is the Next.js server?

Plain-language meaning

Next.js is the web application framework. It supports both the React interface in the browser and JavaScript/TypeScript code running on a server. The browser shows the chat. The server handles operations that should not be entrusted to a web page, including private provider calls, session checks and database access.

In Medix, the server route `frontend/app/(chat)/api/chat/route.ts` receives a chat message. It reads the current session, stores chat state, extracts text and any inline image, and chooses how the request should proceed. This is application orchestration, not the language model itself.

Input	Work performed	Output
Message, history and optional file	Validate the session/message; load history; classify intent; choose current versus new evidence.	A clarification, a general Medix reply, or a request to the evidence pipeline.
Backend evidence and reference context	Run the medical composition module; emit verified answer and evidence events.	A streamed chat response consumed by the React UI.

Suggested speaking script
When I say Next.js server, I mean the server-side part of the website. It is responsible for handling the conversation: who is asking, what they asked before, whether the message is a greeting or a medical question, and which service should handle it. It calls the backend when new video evidence is needed, and later runs the conversational answer layer. The React page is the visible client; the Next.js server is the application coordinator behind it.

Why not call the model directly from the browser?

The current architecture keeps provider credentials and orchestration on the server. It also gives one place to apply session checks, save history, enforce the response workflow and stream the resulting answer.

Are the Next.js server and the medical layer different servers?

No. The medical layer is a set of TypeScript modules invoked by the Next.js chat route. It is a separate responsibility inside the same web server process.

Code trace: frontend/app/(chat)/api/chat/route.ts; frontend/lib/ai/chat-routing.ts; frontend/lib/dev-session.ts

MEDIX / PRESENTATION PREPARATION

B2. What does FastAPI do?

Plain-language meaning

FastAPI is the Python framework used to expose Medix's evidence service over HTTP. It receives structured requests, checks their shape and passes valid questions to the Python pipeline. It is neither a language model nor the website interface. Uvicorn runs the FastAPI application as a network service.

The Python side contains the local processing tools: text embeddings, lexical retrieval, transcript repositories, CLIP frame scoring, FFmpeg extraction and structured backend generation. Keeping these operations in Python lets the existing model and video-processing libraries be composed behind a stable API.

Phase	What FastAPI and its service dependencies do
Receive	Pydantic validates request fields; the text question is stripped and must be non-empty. Readiness checks prevent queries when required components are unavailable.
Acquire evidence	The QA service runs safety triage, video retrieval, temporal localization and evidence extraction. Blocking work is dispatched through a thread pool.
Generate and return	Obtain structured transcript-supported claims, validate their cue references, and serialize an evidence response with artifact URLs.

Suggested speaking script
FastAPI is the entry point to the Python evidence pipeline. Its job is to turn an HTTP request into a call to our retrieval and video-processing services, then turn their result back into structured JSON. The actual medical pipeline sits in service classes behind the route. That distinction matters: the route handles the network contract, while the service layer performs the evidence work.

Why do we have both Next.js and FastAPI?

Next.js handles web sessions, conversation and presentation. FastAPI hosts the Python retrieval and media stack. They communicate through HTTP, so the backend can move from a laptop to Sol without rebuilding the website.

Does FastAPI always return a medical answer?

No. A processed request can return an abstained result when support is insufficient. If the canonical service is not initialized, the endpoint can return a service-unavailable error instead.

Code trace: backend/app.py; backend/application.py; backend/api/routes.py; backend/container.py; backend/services/grounded_qa.py

MEDIX / PRESENTATION PREPARATION

B3. What does POST /api/query mean?

An **API endpoint** is a named network entry point. The address identifies the service, the path identifies the operation, and the HTTP method tells the service what kind of request it is receiving.

Part	Meaning in this project
http://127.0.0.1:5001	The local Python service. 127.0.0.1 means this machine; 5001 is the backend listening port.
POST	Send a request body for processing. Here it starts an evidence-query operation; it is not a static page being opened.
/api/query	The canonical text-question route in FastAPI. The separate Next.js /api/chat route runs on the web application.
Content-Type: application/json	The body is JSON. The required query field contains the non-empty question. It is limited to 2,000 characters by the backend schema.

A concrete request

```
curl -X POST http://127.0.0.1:5001/api/query \
  -H 'Content-Type: application/json' \
  -d '{"query":"How do I check circulation after bandaging?"}'
```

The JSON key `query` is read by QueryRequest. The route calls _run_query, which invokes qa.run. The result is converted into QueryResponse. Do not interpret the URL path as a database table or a model name.

> **Suggested speaking script**
> POST /api/query means that the client is sending a medical question to a specific Python API operation. The question travels as a JSON field, not as a browser page address. The endpoint validates the request, calls the grounded QA pipeline and returns structured evidence. In the website, Next.js makes this call for the user, so the user never has to type a curl command.

Why can a direct API answer look different from the website answer?

The API exposes the backend evidence and its compact validated claims. The website then performs its own conversational generation and verification using that response and applicable checked references.

Does HTTP 200 prove that the answer is correct?

No. It indicates successful HTTP processing. Read the response status and evidence fields: status can be answered or abstained. Neither HTTP status nor a ranking score establishes clinical correctness.

Code trace: backend/api/routes.py: QueryRequest, query, _run_query, QueryResponse

MEDIX / PRESENTATION PREPARATION

B4. What is the evidence response?

The **evidence response** is a structured package sent from FastAPI back to Next.js. It is not merely a paragraph of answer text. It gives the website enough information to compose a response and let the user inspect the supporting source.

Field or group	How to explain it
status, response, claims	Whether the backend answered or abstained; its compact text; individual claims with transcript citation IDs.
citations versus evidence_context	citations support backend claims. evidence_context contains the transcript cues available from the selected interval; the UI should not display all of them automatically.
video_id, clip_url, times	Identity of the source video, URL of the extracted clip, and predicted start/end coordinates in the original recording.
evidence_bundle_id	Provenance identifier tying the question, interval, artifacts and scores together. The evidence manifest also records hashes and extraction metadata.
confidence and safety_disposition	A joint retrieval/localization score and a policy decision. The score is not a probability that the medical answer is true.

Recorded cue example: original time versus clip time

The following reduced contract uses an actual saved test cue. It is not the result of a new request; other fields are omitted. Its extracted clip started at source second 90.

```
{
  "video_id": "NxO5LvgqZe0",
  "start_seconds": 90,
  "evidence_context": [{
    "cue_id": "T015",
    "start_seconds": 168.5,
    "end_seconds": 174.42,
    "clip_start_seconds": 78.5,
    "clip_end_seconds": 84.42
  }]
}
```

Suggested speaking script
Think of the evidence response as a labeled folder. It contains the selected video interval, the transcript cues and their coordinates, the claims that passed backend checks, and links to generated artifacts. Next.js uses this folder as input; it does not simply print every transcript line. For example, source time 168.5 becomes clip time 78.5 when the extracted clip begins at second 90.

Code trace: backend/api/routes.py; backend/evidence/extractor.py; saved response-pipeline source.json

MEDIX / PRESENTATION PREPARATION

B5. The Next.js medical response layer

What does "medical layer" mean?

It is the application code that turns available evidence into the final conversational answer. The main module is `medical-follow-up.ts`. It runs on the Next.js server for initial medical answers as well as follow-ups. It is not another model deployment, another database or an additional Python server.

Step	Purpose
Select	Choose the smallest complete set of directly relevant transcript cues. A cue does not qualify merely because it concerns the same injury.
Augment	Select applicable dated medical summaries separately from video evidence. Use history to resolve the current situation, not as medical proof.
Generate	Ask Qwen for supported claims, a direct answer, practical detail and an optional useful follow-up question. Response length is adapted to intent.
Verify and repair	Check exact citation support, relevance, source scope and completeness. An incomplete draft can receive one further generation/verification cycle.
Render	Add video timestamps only to video-supported claims, use numbered medical links for reference-supported guidance, and keep irrelevant video moments out.

Suggested speaking script
The backend gives us evidence, but evidence is not yet a good conversation. This layer asks the language model to answer the user's actual question in natural language while using that evidence. A second model pass checks whether the claims are supported and useful. If an action is missing, the system can attempt a bounded repair. Finally, code converts approved source IDs into the links and timestamps that the user sees.

Why generate twice: once in Python and again in Next.js?

The backend validates a transcript-grounded evidence artifact; the website adds conversation, checked-reference guidance and patient-facing presentation. This preserves the backend API but duplicates some work and adds model-call latency. It is an architectural trade-off, not a claim of independent clinical review.

Can the verifier still make mistakes?

Yes. The generator and verifier share the Qwen provider. Programmatic ID, quantity, scope and medication checks add constraints, but model agreement does not prove medical accuracy.

Code trace: frontend/lib/ai/medical-follow-up.ts; response-composition.ts; medical-turn-safety.ts

MEDIX / PRESENTATION PREPARATION

B6. What are checked references?

Checked references are **small medical summaries maintained in application code**, each with a source URL, review date and explicit scope. They are separate from retrieved video transcripts. Medix does not perform an unrestricted live web search for every question.

Reference	Scope of the current entry
K001 | NHS cuts and grazes	First-aid dressings and selected warning signs for cuts; not casts, surgical care or prescribed compression.
K002 | St John severe bleeding	Ordinary bleeding-wound dressings and first-aid actions, with restrictions on transferring advice to other contexts.
K003 | NHS burns and scalds	Thermal burn guidance and relevant escalation boundaries; not a generic treatment template for every burn mechanism.
K004 | St John bandages	Ordinary limb bandages and circulation checks; not casts or post-operative dressings.
K005 | NHS injury warning signs	Conditional warning signs after a limb injury; no inferred diagnosis of a sprain, fracture or circulation disorder.

The selector uses topic patterns, exclusions and the active question. It returns at most three entries; an entry becomes ineligible after 180 days without a renewed source check. The current entries are dated 15 October 2026. This is a deliberately limited library, not a comprehensive clinical knowledge service.

> **Suggested speaking script**
> Checked references give the answer layer a separate source for medical guidance that may not be stated in the video. They are short, dated summaries with known URLs and scope limits. A reference-supported statement gets a medical source link, not a borrowed video timestamp. This helps the answer go beyond transcript fragments while keeping provenance visible. The limitation is that we currently cover only a small set of first-aid topics.

Does a checked source mean that every generated sentence is verified by a clinician?

No. The application checks generated claims against the available summaries. The source is trusted within its scope, but source selection, summarization and model interpretation can all fail. Clinical review remains a separate requirement.

Why not let Qwen answer entirely from memory?

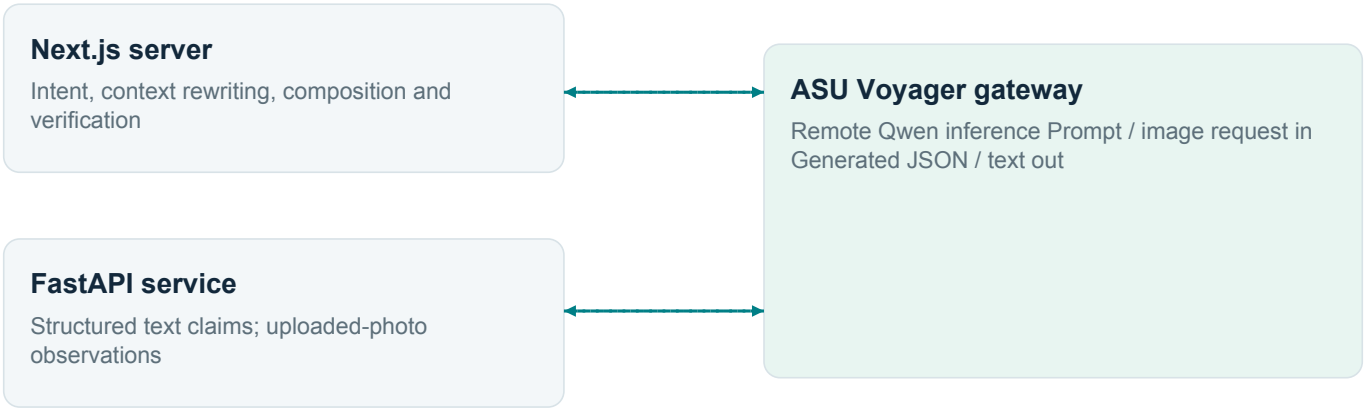
The implemented medical path requires supplied support. When the available video and reference material cannot support a detail, the system should preserve the supported part and communicate the remaining limitation rather than inventing it.

Code trace: frontend/lib/ai/medical-knowledge.ts; medical-follow-up.ts

MEDIX / PRESENTATION PREPARATION

B7. What is the Voyager-Qwen connection?

Voyager is the ASU Research Computing model gateway; Qwen is the model served through it. Medix sends HTTPS requests to `https://openai.rc.asu.edu/v1`. The default alias is `qwen35-27b`. The API format is OpenAI-compatible; that describes the protocol, not the identity of the model.



The connection sends model inputs and receives generated outputs. It does not transmit application code for training. Keys are supplied server-side through the private environment, not exposed as public browser variables.

Model / runtime	Role and location
Voyager Qwen, default qwen35-27b	Remote text generation, bounded image observations and response verification. Both Node and Python clients can invoke it.
all-MiniLM-L6-v2	Local sentence embeddings for catalog, transcript and question relevance. It does not write the final answer.
CLIP ViT-B/32	Local question-to-video-frame similarity. It scores sampled retrieved-video frames; it does not diagnose the uploaded patient image.
Kokoro-82M, optional	Local speech synthesis for the Listen feature; separate from retrieval and medical grounding.

Suggested speaking script
Voyager provides access to the remote Qwen model. Our servers send a prompt, and for a photo observation call they also send a normalized image. Qwen returns generated content through the gateway. The laptop or Sol node still performs the local retrieval and media work. Therefore, running the backend on Sol is different from hosting Qwen there: the shipped Sol job uses CPUs, while inference comes from Voyager.

Does this connection fine-tune the model on our dataset?

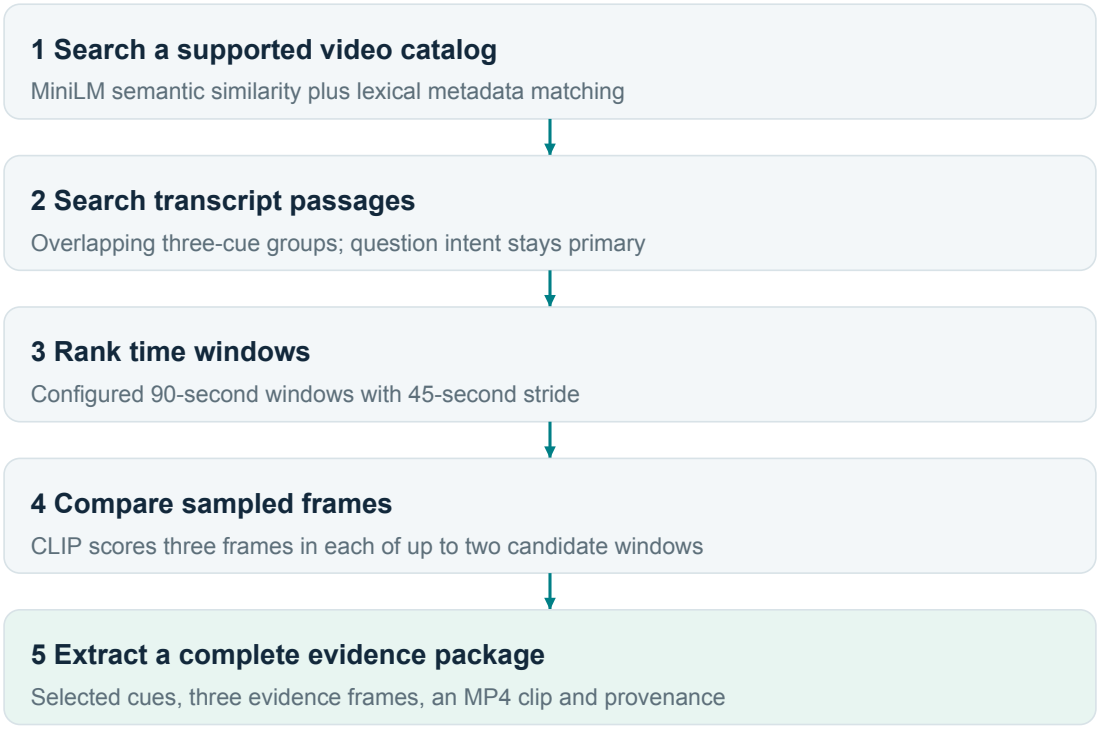
No. The implemented calls request inference. Retrieval changes the context supplied to an existing model; it does not update the model weights.

Code trace: backend/adapters/voyager.py; frontend/lib/ai/providers.ts; backend/adapters/visual_similarity.py

MEDIX / PRESENTATION PREPARATION

B8. RAG and temporal video retrieval

RAG means retrieval-augmented generation: first find relevant material, then supply it as context to a generator. Medix has two complementary evidence sources: retrieved video cues and a small library of applicable checked medical summaries. They are not interchangeable citation types.



Current live passage ranking combines 0.5 lexical coverage + 0.3 dense similarity within each passage, then adds 0.2 metadata score at the video level. Localization uses an effective transcript weight of at least 0.8, with the remainder assigned to CLIP. These are retrieval heuristics, not clinical confidence probabilities.

Suggested speaking script
Retrieval is how the system decides which material is worth showing to the language model. MiniLM represents text meaning numerically; lexical matching preserves important question terms. We then search within a video, not only across video titles. CLIP checks whether sampled frames fit the question, and FFmpeg extracts the resulting interval. The final generator sees transcript evidence, while the frames support localization and the evidence package.

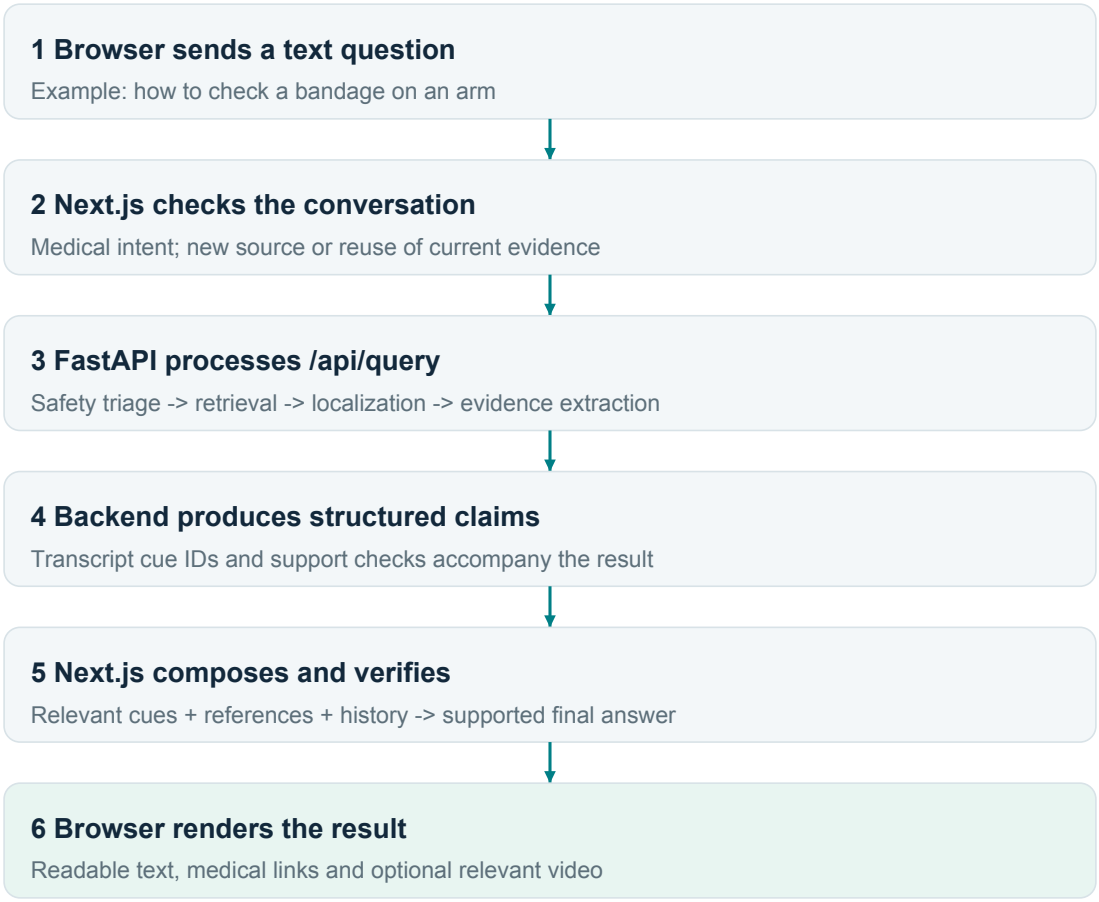
Are the live clip boundaries the dataset answer timestamps?

No. The live path predicts a relevant interval and expands its boundaries when needed to retain complete transcript cues. The evidence manifest explicitly marks that it does not contain gold timestamps. Benchmark annotations are separate evaluation resources.

Code trace: backend/video_retrieval/; temporal_localization/; evidence/extractor.py; research_pipeline.py

MEDIX / PRESENTATION PREPARATION

B9. Text-only: walk one request through



Suggested speaking script
For a new text question, Next.js decides that evidence is needed and sends the question to FastAPI. The backend searches the supported video material, localizes a useful interval and returns validated evidence. The website then generates a conversational answer from selected cues and applicable references. If the next question concerns the same situation, the route can reuse the existing bundle instead of choosing a new video.

Does text-only mean no visual model is used?

No uploaded-photo observation is requested, but the configured localizer still uses CLIP to score frames from retrieved videos. Video-frame relevance and uploaded-image understanding are separate operations.

What happens if the message is just "Hey"?

Small-talk and Medix capability turns bypass medical retrieval. The system should not fabricate a video search or provide unsupported care instructions in that path.

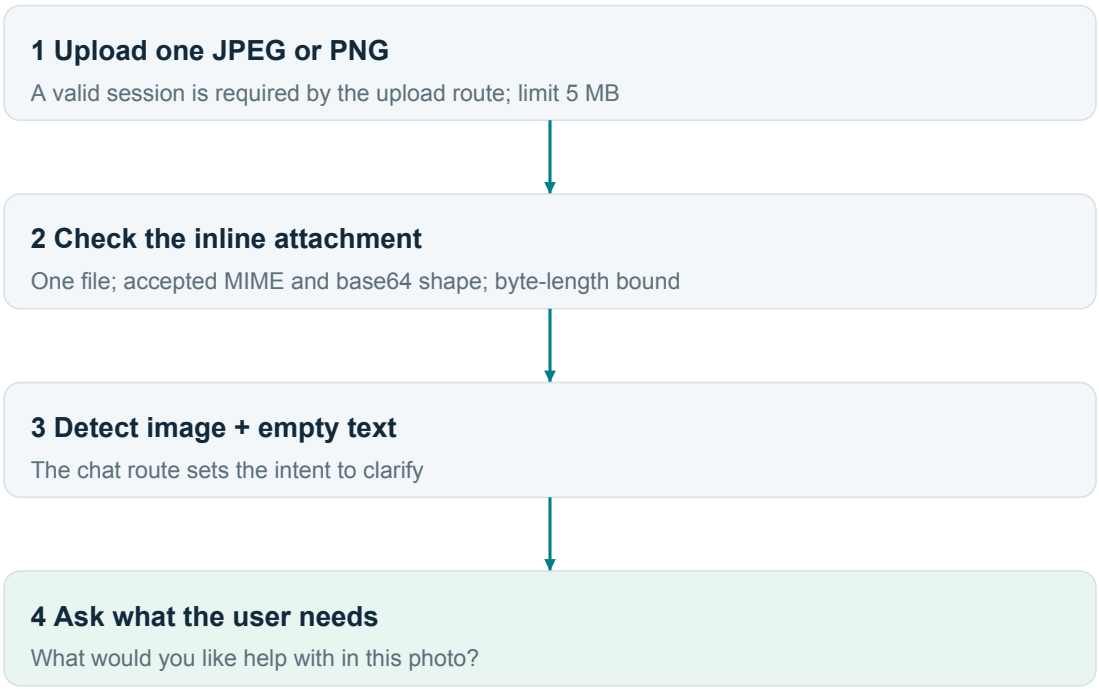
Edit this diagram in Mermaid Live | 02-text.mmd

Code trace: frontend/app/(chat)/api/chat/route.ts; search-procedure.ts; backend/api/routes.py

MEDIX / PRESENTATION PREPARATION

B10. Image only: what really happens

A photo without a text question follows a deterministic clarification path in the website. The system does not automatically choose the procedure, infer injury severity or run the full medical pipeline on that turn.



Suggested speaking script
For image-only input, Medix asks the user what they want help with. That is intentional: seeing a bandage does not tell us whether the user wants application instructions, a tightness check or another explanation. This turn stops before medical retrieval and the vision model. Once the user supplies a clear question, they should attach the photo with that question to enter the combined-input path.

Why does a photo alone not receive an automatic medical answer?

The current route treats essential intent as missing. An image can suggest objects, but it does not reliably identify the user's medical question. Clarification prevents the system from inventing that request.

If the user types a question after uploading the image, is the image automatically reused?

Not by the current medical route. It extracts image bytes from the latest user message only. Reattach the photo with the question when demonstrating the vision path. A later text-only reply can still use conversation/evidence context, but it does not automatically trigger fresh observation of the older photo.

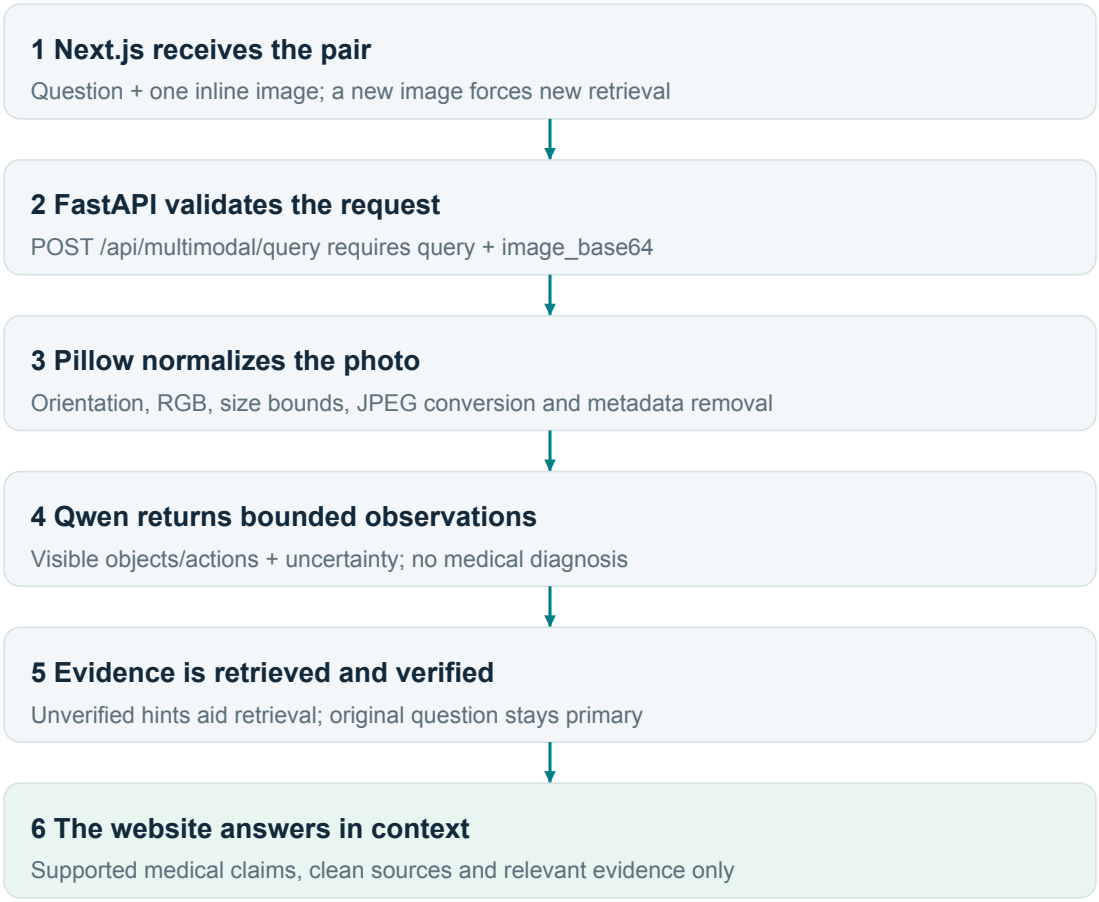
One sentence to remember
Image-only is a supported clarification interaction, not an image-only diagnosis feature.

Edit this diagram in Mermaid Live | 03-image-only.mmd

Code trace: frontend/lib/ai/image-input.ts; app/(chat)/api/files/upload/route.ts; app/(chat)/api/chat/route.ts

MEDIX / PRESENTATION PREPARATION

B11. Text + image: what changes?



The website accepts at most 5 MB. The backend image boundary permits at most 10 MiB and 20 megapixels, rejects multi-frame images, and normalizes a photo to fit within 1024 x 1024 pixels. EXIF is removed from the normalized image transmitted to the model; this does not mean the original uploaded message data has been removed from chat storage.

> **Suggested speaking script**
> In the combined path, the question defines what the user needs, and the image supplies limited visual context. Qwen may identify visible objects or actions, and the application adds those observations as unverified retrieval hints. The original question remains the basis for localization and medical generation. The photo does not become evidence that a bandage is safe, a wound is infected or circulation is normal.

Does the image replace the transcript?

No. The response still needs support from transcript cues or applicable checked references. The API explicitly returns image_context_is_evidence as false.

Do we use the uploaded photo to search video-frame embeddings directly?

The inspected path uses structured Qwen observations to augment text retrieval. CLIP separately compares the question with frames extracted from candidate videos; it is not a direct uploaded-photo-to-video search in this implementation.

Edit this diagram in Mermaid Live | 04-text-image.mmd

MEDIX / PRESENTATION PREPARATION

B12. Follow-ups and the final frontend output

Why the conversation should not restart every turn

The context builder keeps the active procedure, the latest usable evidence output and recent text. Same-topic follow-ups generally continue with that bundle. An explicit new source, clear topic change or new image can lead to fresh retrieval. A model-reported topic mismatch inside a committed evidence turn is handled conservatively rather than silently switching to an unrelated clip.

User sees	What the implementation is doing
A direct answer and practical detail	Render approved claims in a readable order; use adaptive ceilings for brief factual questions versus failed checks or broader explanations.
A clickable timestamp	Map an approved video-supported claim onto a real cue interval; dispatch a seek event to the corresponding evidence workspace.
A medical source number	Link to the fixed reference that supports that statement. It does not claim that the fact was spoken in the video.
A relevant transcript and evidence moments	Render the selected cues, not every cue from a broadly related clip. Skip unrelated gaps in native clip playback and bound selected ranges.
Progress followed by streamed text	Medical composition waits for verification before progressively writing the answer. Progress text does not mean draft medical claims are being streamed unchecked.

Suggested speaking script
The final interface is meant to make the answer and its support easy to inspect. A video timestamp takes the user to a specific moment. A numbered medical link points to a separate reference. Follow-ups can reuse the same evidence, which helps preserve the situation and avoids attaching another broad video just because the user asked a short question. The response should answer the new intent, not repeat the whole original procedure.

Why can a narrow follow-up be just one sentence?

Length should match intent. A duration question can be answered briefly, while a failed check needs a next action and any relevant supported escalation. Shortness alone is not the quality criterion.

Are timestamp synchronization and browser playback already fully validated?

Range and coordinate logic have regression coverage, but continuous playback in the earlier in-app browser was not fully confirmed. Demonstrate it in the normal browser and distinguish a correct seek coordinate from proof of frame-exact playback.

Code trace: query-contextualizer.ts; medical-follow-up.ts; evidence-timestamps.ts; components/chat/{message,procedure-card}.tsx

MEDIX / PRESENTATION PREPARATION

C1. Three demo scripts to rehearse

Demo 1 | Text and a contextual follow-up

Type: "I just put a bandage on someone's arm. How do I know if it's too tight?" Then ask: "What if the colour doesn't come back quickly?"

Observe: the answer should explain the check and an appropriate supported action; medical links and relevant video support should be distinguishable. The follow-up should respond to the failed-check situation without selecting unrelated moments. Exact wording and completeness can vary.

> **Say while demonstrating**
> I am checking whether the system answers a caregiver's practical question, uses the previous situation and keeps the supporting evidence relevant. This is an interaction test, not a clinical validation result.

Demo 2 | Image-only clarification

Do: upload a small, non-identifying JPEG or PNG of a bandaged forearm, leave text blank and send. **Expected:** "What would you like help with in this photo?"

> **Say while demonstrating**
> This branch does not infer what the user wants or diagnose the photo. The model has not performed a medical image assessment here; the application is asking for the missing question.

Demo 3 | Text plus the same image

Do: reattach the photo and ask: "I just put this bandage on someone's arm. How can I check whether it's too tight?"

Observe: the photo can supply a visible-bandage context, but the answer must not claim to determine circulation from it. Look for supported guidance and a relevant timestamp or medical source link. This documentation task has not newly validated an actual uploaded-photo inference run.

> **Say while demonstrating**
> The photo guides what material is relevant, while the medical advice still needs evidence. That separation is the important design choice to look for in this demo.

If a live call is slow: explain the pipeline while the progress indicator is visible. If a service fails, show the error honestly and use the architecture diagram or saved evaluation examples. Never describe an expected answer as a result that actually ran.

MEDIX / PRESENTATION PREPARATION

C2. Five-minute run of show

Clock	Point to	What to say or do
0:00-0:35	Page 2	State the problem: useful guidance is scattered across videos; transcript copying and broad matches are not enough. Introduce evidence-grounded conversational assistance.
0:35-1:20	Page 3	Trace the central graph once. Next.js coordinates; FastAPI acquires evidence; the response layer creates the final conversation.
1:20-2:00	Pages 4-7	Define server, API endpoint and evidence response. Explain that POST /api/query returns a structured package, not the whole patient-facing experience.
2:00-2:40	Pages 8-10	Explain generation plus verification, checked references and the Voyager/Qwen connection. Distinguish model service, model and local processing.
2:40-3:25	Pages 12-14	Explain the three modes. Text retrieves; image-only clarifies; text + image adds bounded observations before evidence retrieval.
3:25-4:25	Page 16 and live website	Run the text question and one follow-up; click one relevant timestamp. Briefly show photo-only clarification if time permits.
4:25-5:00	Page 21	State the limits: partial answers, source coverage, provider dependence, no weight training and no clinical validation claim. Invite questions.

Opening script

Suggested speaking script
Medix addresses a gap between long instructional medical videos and short conversational help. A user should be able to ask a practical question, get an understandable answer and inspect the relevant support. I will show how the website coordinates a Python evidence pipeline and remote Qwen inference, then demonstrate how the same evidence can support a follow-up.

Closing script

Suggested speaking script
The current implementation combines retrieval, temporal localization, bounded visual context and claim verification. Its strengths are inspectable provenance and a conversation-aware workflow. Its main limitations are restricted source coverage, variable model quality and latency, and the absence of independent clinical validation. The next evaluation should examine complete multi-turn behavior, citation faithfulness and real multimodal cases, not only whether an endpoint returns a valid response.

One sentence to remember
During the graph explanation: point to a box, state its job, say what it receives, and say what it returns.

MEDIX / PRESENTATION PREPARATION

C3. Likely questions: system architecture

1. What exactly is your project contribution?

The implemented contribution is the application pipeline: retrieve medical video evidence, localize a relevant interval, retain provenance, and compose a contextual answer with source-aware verification. It is not a new foundation model. Any research novelty or superiority claim needs a separate, clearly defined evaluation.

2. Is Next.js only the frontend?

It includes the frontend UI and a server-side application layer. Medix uses server routes for sessions, history, intent, backend calls, provider calls and response streaming.

3. Is FastAPI the AI model?

No. It is the Python HTTP framework. The services behind it use local retrieval models and call Voyager for Qwen inference.

4. What is the difference between /api/chat and /api/query?

/api/chat is the Next.js conversation route. /api/query is the FastAPI evidence-query route. They live in different services and serve different responsibilities, even though both contain /api in the path.

5. Why is the evidence response a separate box?

It is a useful boundary contract: the backend returns claims, transcript context, timing and provenance. The website can then decide how to produce and display a conversation without exposing every raw cue.

6. Is the medical response layer another microservice?

No. It is code invoked inside Next.js. Its main entry is generateVerifiedFollowUp, which handles initial medical composition as well as follow-ups.

7. Where is the model running?

The default Qwen text/vision inference is accessed through ASU Voyager. MiniLM, CLIP, transcript processing and FFmpeg run with the backend. The frontend also makes Voyager calls for its own orchestration and response tasks.

8. Why not merge everything into one service?

The current split reuses Python model/media libraries and a Next.js chat stack, and allows independent backend hosting. It also creates two generation stages, more network boundaries and latency. A future simplification should preserve the provenance and verification contracts.

MEDIX / PRESENTATION PREPARATION

C4. Likely questions: evidence and medical safety

1. Is Medix trained or fine-tuned on MedVidQA?

No training or fine-tuning occurs in the selected live path. MedVidQA supplies data for retrieval preparation and research evaluation. The active system supplies context to existing models at inference time.

2. Are you using dataset answer labels to find the clip?

The runtime catalog uses title/author metadata and cached transcript content. The live evidence extractor records contains_gold_timestamps as false. Predicted boundaries should not be presented as ground-truth annotation boundaries.

3. How do you know the medical answer is correct?

We can check source support, citation identity, relevance, constraints and selected coverage tests. These do not certify medical correctness. Clinician review, broader cases and independent evaluation remain necessary.

4. Does a matching transcript prove the answer is safe?

No. A transcript may be incomplete, outdated or unsuitable for the current situation. The response layer prefers applicable checked guidance, checks scope and conflicts, and may withhold unsupported details.

5. What is the difference between video and medical citations?

A video timestamp supports what is stated in a specific cue interval. A reference link supports guidance from the checked summary. An extra medical claim must not inherit a timestamp from an unrelated video sentence.

6. Why can the system say it has only partial evidence?

Some supported claims can be useful even when another requested aspect is missing. The system can keep the verified portion rather than fabricate the rest. A partial result is a limitation to report, not a successful full answer.

7. Does the image tell you whether the bandage is too tight?

No. The implemented observation step is limited to visible objects/actions and uncertainty. The application should not infer circulation or tightness from appearance alone.

8. Are source dates automatically refreshed?

No. The entries have stored checked dates and a 180-day eligibility rule. Maintaining and expanding those summaries requires a real source review; the runtime does not silently browse and refresh them.

MEDIX / PRESENTATION PREPARATION

C5. Likely questions: demo behavior and deployment

1. Why is the first response sometimes slow?

Startup builds or loads local model features. A new query can include retrieval, multiple candidate windows, media extraction, backend generation, frontend generation, verification and one repair. Provider latency or queueing adds to this. Progress indicators do not establish a speed guarantee.

2. What does Sol do if Voyager already provides the model?

Sol hosts the Python evidence/API work and its local resources. The shipped job requests 4 CPUs and 16 GB for two hours, without a GPU. Voyager remains a separate inference service.

3. What does the SSH tunnel do?

It forwards a local Mac port to the allocated compute node's backend port. With local 5002 forwarding to node:5001, the frontend can use MEDIX_API_URL=http://127.0.0.1:5002. It does not start the backend or allocate a node.

4. What does PENDING in squeue mean?

The scheduler has not assigned running resources. It is not proof of a Python or frontend error. Wait for RUNNING and a compute node before expecting the tunnel to reach a live service.

5. Can the system work without videos downloaded locally?

The active configuration needs local video bytes for CLIP frame extraction and FFmpeg clips. An on-demand repository exists in the code, but it is not enabled by the inspected runtime defaults. The demo should not claim that remote streaming alone replaces the local evidence path.

6. What if the API key, cache or provider is unavailable?

Missing initialization resources can make the backend unready; provider failures can interrupt generation; unavailable evidence can cause abstention. Check health, private environment settings and logs, and distinguish readiness from correctness.

7. What should you say if the live answer repeats itself?

Describe it as a remaining quality issue. Recent changes add context, completeness checks and a bounded repair, but generation still varies. Do not claim that a single successful demo establishes consistent clinical performance.

> **One sentence to remember**
> Demonstrate what ran, explain what the code intends, and keep remaining limitations separate from measured results.

Before startup: use the existing private backend/.env and frontend/.env.local files, migrated PostgreSQL tables, local evidence caches and cached models. Stop an older owned server before reusing its port. For Sol, verify the supplied account association is valid and wait for RUNNING before the compute-node lookup and tunnel on page 24.

MEDIX / PRESENTATION PREPARATION

D1. Quick sheet and implementation map

Term	One-line explanation
Next.js server	The web application coordinator: sessions, conversation, routing, provider calls and streaming.
FastAPI service	The Python network boundary around retrieval, video processing and backend evidence generation.
Evidence response	Structured backend output with claims, transcript cues, timings, artifact URLs and provenance.
Medical response layer	Server-side Next.js modules that generate and verify the conversational answer.
Voyager / Qwen	Remote ASU model gateway / default model served by that gateway.
RAG / MiniLM / CLIP	Evidence-augmented generation / local text embeddings / local video-frame relevance scoring.
Source time / clip time	Original recording coordinate / offset within the extracted video; clip time = source time - clip start.
Checked reference	A dated, scope-restricted summary with a fixed medical source URL; not a live web search.

Where to point in the code

Web orchestration: frontend/app/(chat)/api/chat/route.ts; frontend/lib/ai/chat-routing.ts

Composition and sources: frontend/lib/ai/medical-follow-up.ts; response-composition.ts; medical-knowledge.ts

Evidence bridge: frontend/lib/ai/tools/search-procedure.ts; tools/medix-api-client.ts

Image boundaries: frontend/lib/ai/image-input.ts; backend/adapters/structured_vision.py; vision_context.py

Python API and composition: backend/api/routes.py; backend/container.py; backend/research_pipeline.py

Retrieval and artifacts: backend/video_retrieval/; temporal_localization/; evidence/extractor.py; video_processing/

Models and UI: backend/adapters/voyager.py; frontend/lib/ai/providers.ts; components/chat/procedure-card.tsx

Launch and recorded evidence: backend/medix_live_backend.sbatch; development/evaluation/results/response-pipeline-2026-10-15/

Claims to avoid: "fine-tuned on our data", "the model watched the whole video", "the image confirms circulation", "confidence equals clinical accuracy", or "all test cases are fully answered". Each would overstate the current implementation.

MERMAID SOURCE / COMPANION FILES INCLUDED

D2. Editable diagrams: overall and text

The following Mermaid sources describe the same logical connections as the print diagrams. Open a preloaded editor link, or paste the source into Mermaid Live Editor. The print layout is optimized separately for reading.

Overall system

Edit this diagram in Mermaid Live | 01-overall.mmd

```
flowchart TD
 U[Browser: text or image] --> N[Next.js /api/chat]
 P[(PostgreSQL chat history)] <--> N
 N --> R{Route and resolve context}
 R -->|clarify or general| O[Streamed chat output]
 R -->|new medical question| F[FastAPI canonical QA]
 R -->|same-topic follow-up| C[Next.js response composer]
 D[(Catalog + transcripts + local videos)] --> F
 V[Voyager Qwen: text and vision] <--> F
 F --> E[Evidence bundle + validated backend claims]
 E --> C
 K[Checked NHS / St John references] --> C
 V <--> C
 C --> O
 E --> M[Optional relevant clip + transcript]
 M --> U
 O --> U
```

Text-only input

Edit this diagram in Mermaid Live | 02-text.mmd

```
flowchart TD
 A[Text question] --> B[Validate session and message]
 B --> C[Intent + conversation context]
 C -->|new source| D[POST /api/query]
 C -->|same topic| I[Reuse current evidence]
 D --> E[Safety triage]
 E --> F[MiniLM + lexical passage retrieval]
 F --> G[Transcript windows + CLIP frame scoring]
 G --> H[Extract cues, frames and clip]
 H --> J[Voyager structured backend claims]
 J --> I
 I --> K[Select relevant cues + checked references]
 K --> L[Generate, verify and bounded repair]
 L --> M[Chat + inline sources + optional video]
```

Companion sources: `development/docs/medix-presentation-prep-diagrams/`. Editor links contain diagram text only.

MERMAID SOURCE / WHAT ACTUALLY RUNS

D3. Editable diagrams: the image modes

Image-only clarification

Edit this diagram in Mermaid Live | 03-image-only.mmd

```
flowchart TD
 A[One JPEG or PNG, no text] --> B[Validate upload and inline bytes]
 B --> C[Next.js detects empty question + image]
 C --> D[Deterministic clarification reply]
 D --> E[What would you like help with in this photo?]
 E --> F[User provides a specific question]
 F --> G[Reattach photo with question for vision]
 G --> H[Text + image workflow]
 C -.-> X[No vision call or RAG for this turn]
 C -.-> Y[No diagnosis or automatic procedure choice]
```

Text + image

Edit this diagram in Mermaid Live | 04-text-image.mmd

```
flowchart TD
 T[Text question + history] --> J[Next.js route and contextualize]
 I[One inline JPEG or PNG] --> J
 J --> A[POST /api/multimodal/query]
 A --> P[Pillow: validate, orient, resize, strip EXIF]
 P --> V[Voyager Qwen visual observation]
 V --> O[Validated objects / actions / uncertainty]
 O --> Q[Question + unverified retrieval hints]
 Q --> R[MiniLM + lexical catalog / transcript retrieval]
 T --> L[Original question for localization and QA]
 R --> L
 L --> C[CLIP video frames + temporal localization]
 C --> E[Transcript cues + frames + extracted clip]
 E --> B[Backend structured text claims + validation]
 B --> G[Next.js medical-reference generation + verification]
 O -.->|context, not clinical evidence| G
 G --> U[Streamed answer + precise relevant evidence]
```

How this guide was prepared

The active request route, model adapters, retrieval/localization services, evidence schemas, reference selector, UI evidence renderer and launch script were read directly. The supplied prep guide was used only to organize the material into scripts, takeaways and a question bank. Unrelated paper content was not transferred to this project.

Existing local resource and evaluation artifacts were used as implementation evidence. The recorded cue on page 7 is a reduced example from the saved test source, not a newly generated or invented timestamp. No new clinical accuracy experiment or actual photo inference test was run to prepare this guide.

The earlier architecture document remains available separately. This document adds the plain-language explanations and rehearsal material; it does not change application behavior.

PREPARED ENVIRONMENTS / TERMINAL COMMANDS ONLY

D4. Exact launch commands

Local Mac | backend terminal

```
cd /Users/sairithwikk/Documents/Medix
source /Users/sairithwikk/.cache/medix/venv/bin/activate
HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1 \
python -m uvicorn backend.app:app --host 127.0.0.1 \
  --port 5001 --env-file backend/.env
```

Local Mac | frontend terminal

```
cd /Users/sairithwikk/Documents/Medix/frontend
MEDIX_API_URL=http://127.0.0.1:5001 pnpm dev
```

Sol backend | submit, inspect RUNNING node, then tunnel from Mac

```
LIVE_JOB=$(ssh skukunu1@sol.asu.edu \
  'cd ~/Medix && module purge && module load mamba/latest &&
   source activate medix &&
   sbatch --parsable --account=class_cse57268549fall2026 \
   backend/medix_live_backend.sbatch')
ssh skukunu1@sol.asu.edu \
  "squeue -j $LIVE_JOB -o '%.18i %.8T %.20N'"
ssh skukunu1@sol.asu.edu \
  "tail -n 30 ~/Medix/medix-live-$LIVE_JOB.out"
LIVE_NODE=$(ssh skukunu1@sol.asu.edu \
  "squeue -h -j $LIVE_JOB -t R -o %N")
test -n "$LIVE_NODE" && ssh -N \
  -L "5002:$LIVE_NODE:5001" skukunu1@sol.asu.edu
```

Sol backend | separate Mac frontend terminal

```
curl -fsS http://127.0.0.1:5002/api/health
cd /Users/sairithwikk/Documents/Medix/frontend
MEDIX_API_URL=http://127.0.0.1:5002 pnpm dev
```